COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	September 10, 2026	**DATE SUBMITTED:**	September 15, 2026
NAME:	Carlo Mari B. Asajar	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 **SORTING ALGORITHMS PART 2**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of shell, merge and quick sort.

Preparation Task:

- Create an array of elements with random values. The array must contain 100 elements that are not sorted.
- Use the created array for each sorting algorithm. Create a header file for the implementation of the different sorting algorithms.
- Import this header file into your main.cpp.

Show the output of the preparation task in the table.
(Table 8-1. Array of Values for Sort Algorithm Testing)

Screenshot of Code	**My Code: (main.cpp)** ILOA > main.cpp > ... 1 #include <iostream> 2 #include <cstdlib> 3 #include <ctime> 4 #include "sortingAlgoPart2.h" 5 6 const int maxSize = 100; 7 8 void displayArr(int arr[]); 9 10 int main() { 11 12 // Create an array with 100 unsorted elements 13 int unsortedArr[maxSize]; 14 15 // Use time(0) as seed for random values 16 srand(time(0));

<table>
<tr><td></td><td>

```
17
18      // Populate the array with random values
19      for (int i = 0; i < maxSize; i++) {
20          unsortedArr[i] = rand() % 100;
21      }
22
23      // Display the original array
24      std::cout << "\nArray:" << std::endl;
25      std::cout << "---------------------------------" << std::endl;
26      displayArr(unsortedArr);
27      std::cout << std::endl;
28  }
29
30  // Display all elements in the array
31  void displayArr(int arr[]) {
32      for (int i = 0; i < maxSize; i++) {
33          std::cout << arr[i] << " ";
34      }
35      std::cout << std::endl;
36  }
```

</td></tr>
<tr><td>Output</td><td>

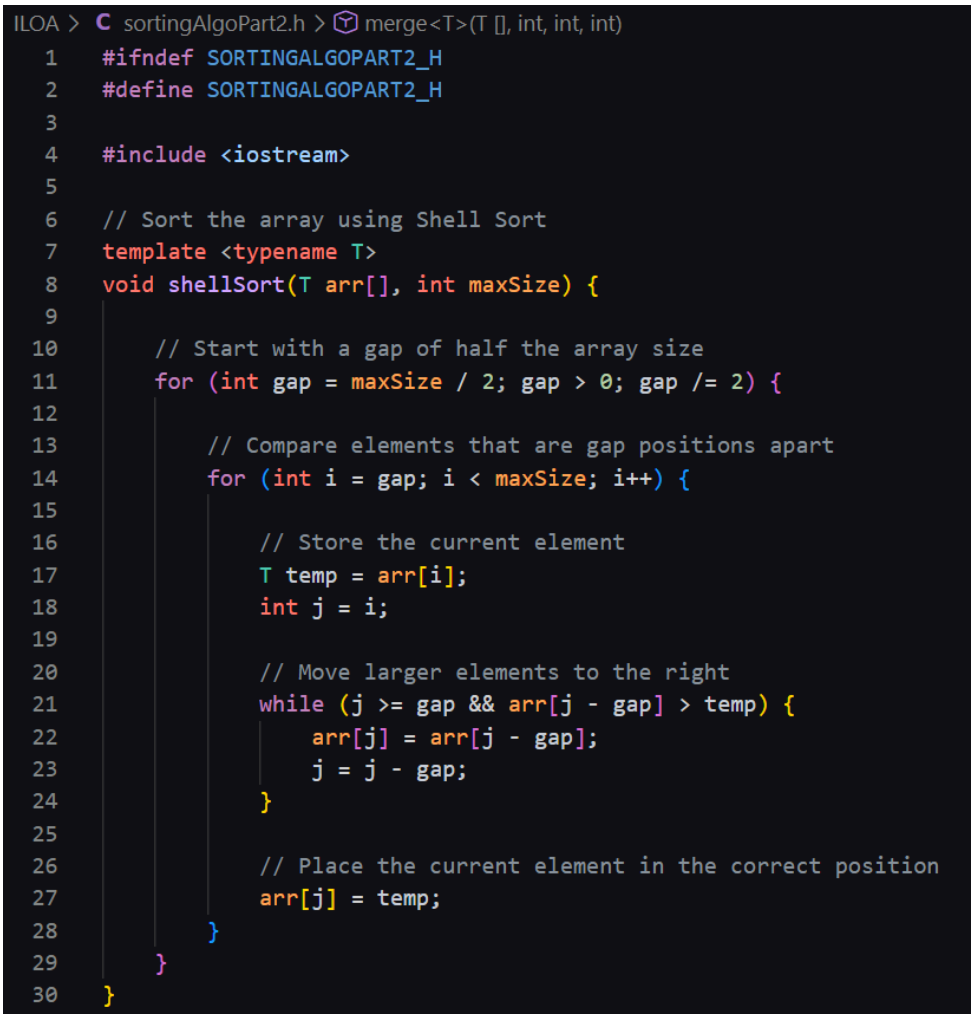
```
Array:
---------------------------------
24 5 61 82 7 0 14 91 68 26 10 58 34 31 88 40 3
5 64 21 4 33 68 29 65 97 54 23 46 45 73 70 89
60 61 76 69 15 60 32 64 82 85 23 1 24 23 38 37
 31 1 71 25 92 48 21 83 86 10 25 76 64 94 26 2
9 47 96 91 85 36 3 16 59 47 32 47 4 10 0 0 42
45 68 30 62 65 96 19 43 49 90 29 29 11 49 75 7
7 52 74 40 57
```

</td></tr>
<tr><td>Observations</td><td>The code successfully created 100 random values using rand() % 100, then stored the values in unsortedArr[maxSize]. The for loop was used to fill all 100 positions, while displayArr(unsortedArr) showed the values in their original unsorted order. Some numbers can appear more than once because random values from 0 to 99 are allowed. Overall, the output showed that the array was prepared properly and can now be used for the different sorting algorithms.</td></tr>
</table>

Table 8-1. Array of Values for Sort Algorithm Testing

A.1. Shell Sort

Include a screenshot of your code and output console along with your observations in the table.
(Table 8-2. Shell Sort Technique)

Screenshot of Code	**My Code: (sortingAlgoPart2.h)** 

```
ILOA > C sortingAlgoPart2.h > merge<T>(T [], int, int, int)
#ifndef SORTINGALGOPART2_H
#define SORTINGALGOPART2_H

#include <iostream>

// Sort the array using Shell Sort
template <typename T>
void shellSort(T arr[], int maxSize) {

    // Start with a gap of half the array size
    for (int gap = maxSize / 2; gap > 0; gap /= 2) {

        // Compare elements that are gap positions apart
        for (int i = gap; i < maxSize; i++) {

            // Store the current element
            T temp = arr[i];
            int j = i;

            // Move larger elements to the right
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j = j - gap;
            }

            // Place the current element in the correct position
            arr[j] = temp;
        }
    }
}
```

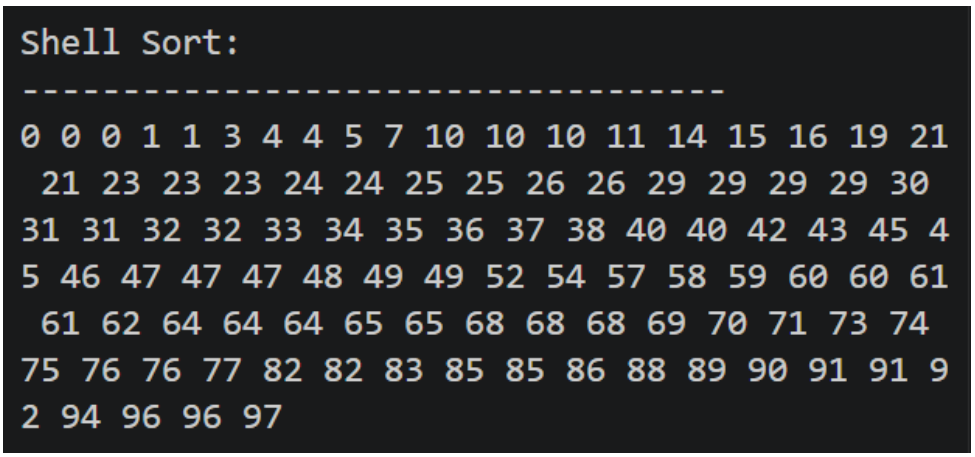
	My Code: (main.cpp)
```
38      // Sort the array using Shell Sort
39      std::cout << "\nShell Sort:" << std::endl;
40      std::cout << "----------------------------------" << std::endl;
41      shellSort(shellArr, maxSize);
42      displayArr(shellArr);
43
```
|
| Output |  |
| Observations | The code successfully sorted the 100 random values using the ***shellSort(shellArr, maxSize)*** function, where the values were gradually arranged through comparisons using different gaps. The ***shellArr*** was first copied from ***unsortedArr***, so the original random values stayed unchanged for comparison with the other sorting methods. After running ***shellSort()***, the ***displayArr(shellArr)*** function showed the values arranged from smallest to largest, showing that the sorting process worked properly. Overall, the values were first arranged in smaller groups and then became more organized as the gap became smaller until the final sorted order was reached. |

Table 8-2. Shell Sort Technique

A.2. Merge Sort

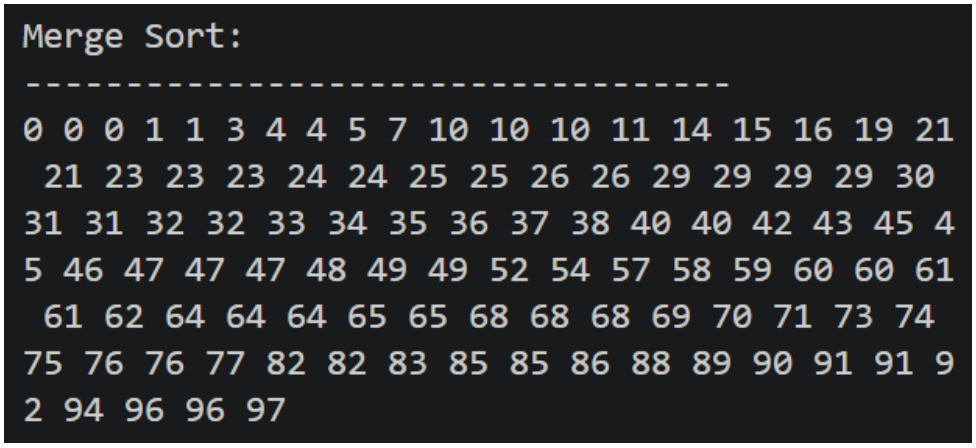
Include a screenshot of your code and output console along with your observations in the table.
(Table 8-3. Merge Sort Algorithm)

Screenshot of Code	**My Code: (sortingAlgoPart2.h)**
	
	`// Merge two sorted parts of the array`
	`template <typename T>`
	`void merge(T arr[], int left, int middle, int right) {`
	
	`    // Set the starting positions of the two parts`
	`    int i = left;`
	`    int j = middle + 1;`
	`    int k = 0;`
	
	`    // Calculate the size of the temporary array`
	`    int maxSize = right - left + 1;`
	
	`    // Create a temporary array`
	`    T* temp = new T[maxSize];`
	
	`    // Compare elements from both parts`
	`    while (i <= middle && j <= right) {`
	`        if (arr[i] <= arr[j]) {`
	`            temp[k] = arr[i];`
	`            i++;`
	`        }`
	`        else {`
	`            temp[k] = arr[j];`
	
	`            j++;`
	`        }`
	
	`        k++;`
	`    }`
	
	`    // Copy the remaining elements from the left part`
	`    while (i <= middle) {`
	`        temp[k] = arr[i];`

```
64          i++;
65          k++;
66      }
67
68      // Copy the remaining elements from the right part
69      while (j <= right) {
70          temp[k] = arr[j];
71          j++;
72          k++;
73      }
74
75      // Copy the sorted elements back to the original array
76      for (int x = 0; x < maxSize; x++) {
77          arr[left + x] = temp[x];
78      }
79
80      // Delete the temporary array
81      delete[] temp;
82  }
```

```
83
84  // Sort the array using Merge Sort
85  template <typename T>
86  void mergeSort(T arr[], int left, int right) {
87
88      // Check if the array can still be divided
89      if (left < right) {
90
91          // Find the middle position
92          int middle = (left + right) / 2;
93
94          // Sort the left part of the array
95          mergeSort(arr, left, middle);
96
97          // Sort the right part of the array
98          mergeSort(arr, middle + 1, right);
99
100         // Merge the two sorted parts
101         merge(arr, left, middle, right);
102     }
103 }
```

	My Code: (main.cpp)
```
    // Sort the array using Merge Sort
    std::cout << "\nMerge Sort:" << std::endl;
    std::cout << "----------------------------------" << std::endl;
    mergeSort(mergeArr, 0, maxSize - 1);
    displayArr(mergeArr);

```
| Output | 

```
Merge Sort:
----------------------------------
0 0 0 1 1 3 4 4 5 7 10 10 10 11 14 15 16 19 21
 21 23 23 23 24 24 25 25 26 26 29 29 29 29 30
31 31 32 32 33 34 35 36 37 38 40 40 42 43 45 4
5 46 47 47 47 48 49 49 52 54 57 58 59 60 60 61
 61 62 64 64 64 65 65 68 68 68 69 70 71 73 74
75 76 76 77 82 82 83 85 85 86 88 89 90 91 91 9
2 94 96 96 97
```
| Observations | The code successfully sorted the 100 random values using the ***mergeSort(mergeArr, 0, maxSize - 1)*** function, where the values were divided into smaller parts and then combined again in sorted order. The ***merge()*** function compared the values from the left and right parts using ***arr[i] <= arr[j]***, while the temporary array was used to arrange the values before putting them back into the original array. The ***mergeArr*** was copied from ***unsortedArr***, so the original random values stayed unchanged for comparison with the other sorting methods. After running ***mergeSort()***, the ***displayArr(mergeArr)*** function showed the values arranged from smallest to largest, showing that the sorting process worked properly. Overall, the values were first separated into smaller groups and then combined step by step until the final sorted order was reached. |

Table 8-3. Merge Sort Algorithm

A.3. Quick Sort

Include a screenshot of your code and output console along with your observations in the table 8-4.

(Table 8-4. Quick Sort Algorithm)

Screenshot of Code	**My Code: (sortingAlgoPart2.h)**
	
	105 // Partition the array for Quick Sort
	106 template <typename T>
	107 int partition(T arr[], int low, int high) {
	108
	109 // Select the last element as the pivot
	110 T pivot = arr[high];
	111
	112 // Set the position before the first element
	113 int i = low - 1;
	114
	115 // Compare each element with the pivot
	116 for (int j = low; j < high; j++) {
	117
	118 // Move smaller elements before the pivot
	119 if (arr[j] < pivot) {
	120 i++;
	121
	122 // Swap the elements
	123 T temp = arr[i];
	124 arr[i] = arr[j];
	125 arr[j] = temp;
	126 }
	127 }
	
	128
	129 // Place the pivot in its correct position
	130 T temp = arr[i + 1];
	131 arr[i + 1] = arr[high];
	132 arr[high] = temp;
	133

```
134         // Return the pivot position
135         return i + 1;
136     }
137
138     // Sort the array using Quick Sort
139     template <typename T>
140     void quickSort(T arr[], int low, int high) {
141
142         // Check if the array can still be divided
143         if (low < high) {
144
145             // Find the pivot position
146             int pivot = partition(arr, low, high);
147
148             // Sort the elements before the pivot
149             quickSort(arr, low, pivot - 1);
150
151             // Sort the elements after the pivot
152             quickSort(arr, pivot + 1, high);
153         }
154     }
```

My Code: (main.cpp)

```
50      // Sort the array using Quick Sort
51      std::cout << "\nQuick Sort:" << std::endl;
52      std::cout << "---------------------------------" << std::endl;
53      quickSort(quickArr, 0, maxSize - 1);
54      displayArr(quickArr);
55
56      std::cout << std::endl;
57  }
58
```

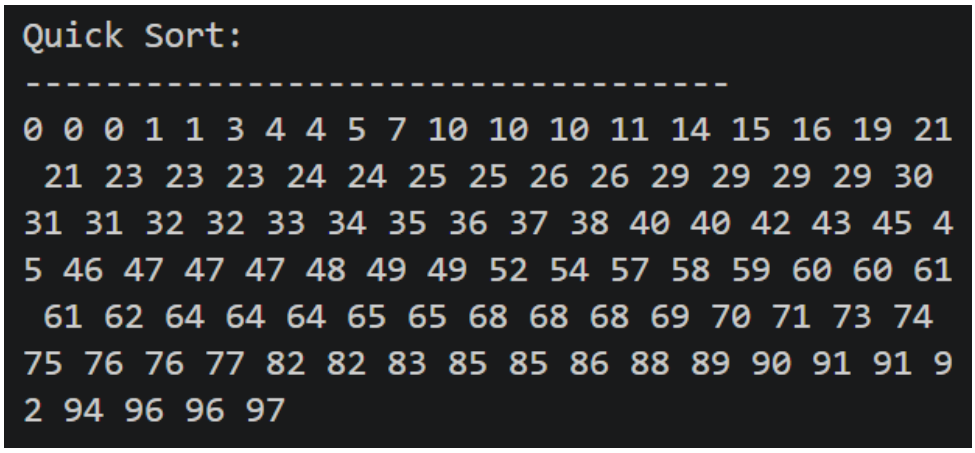
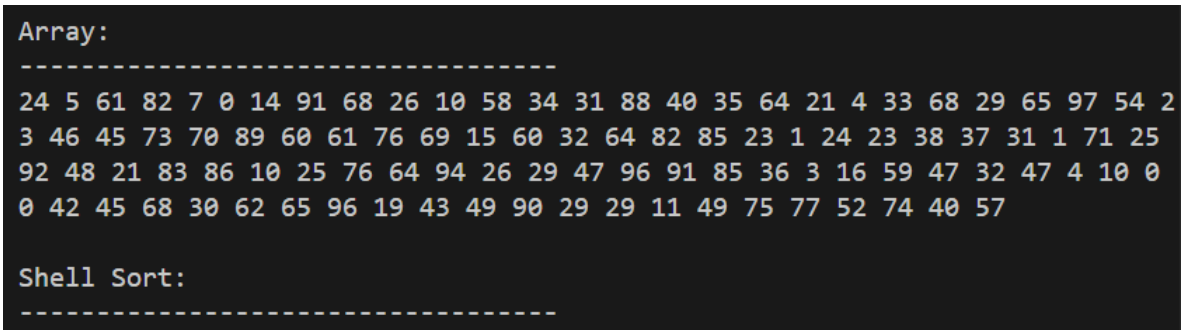
Output	
Observations	The code successfully sorted the 100 random values using the ***quickSort(quickArr, 0, maxSize - 1)*** function, where the values were divided into smaller parts based on a selected pivot. The ***partition()*** function used ***arr[high]*** as the pivot and compared the other values using ***arr[j] < pivot***, moving smaller values before the pivot while the larger values stayed after it. The ***quickArr*** was copied from ***unsortedArr***, so the original random values stayed unchanged for comparison with the other sorting methods. After running ***quickSort()***, the ***displayArr(quickArr)*** function showed the values arranged from smallest to largest, showing that the sorting process worked properly. The values were divided and sorted step by step around the pivot until the final sorted order was reached.

Table 8-4. Quick Sort Algorithm

My Output: (Complete)

Part 1:



Part 2:

```
0 0 0 1 1 3 4 4 5 7 10 10 10 11 14 15 16 19 21 21 23 23 23 24 24 25 25 26 2
6 29 29 29 29 30 31 31 32 32 33 34 35 36 37 38 40 40 42 43 45 45 46 47 47 4
7 48 49 49 52 54 57 58 59 60 60 61 61 62 64 64 64 65 65 68 68 68 69 70 71 7
3 74 75 76 76 77 82 82 83 85 85 86 88 89 90 91 91 92 94 96 96 97

Merge Sort:
-----------------------------------
0 0 0 1 1 3 4 4 5 7 10 10 10 11 14 15 16 19 21 21 23 23 23 24 24 25 25 26 2
6 29 29 29 29 30 31 31 32 32 33 34 35 36 37 38 40 40 42 43 45 45 46 47 47 4
7 48 49 49 52 54 57 58 59 60 60 61 61 62 64 64 64 65 65 68 68 68 69 70 71 7
3 74 75 76 76 77 82 82 83 85 85 86 88 89 90 91 91 92 94 96 96 97

Quick Sort:
-----------------------------------
0 0 0 1 1 3 4 4 5 7 10 10 10 11 14 15 16 19 21 21 23 23 23 24 24 25 25 26 2
6 29 29 29 29 30 31 31 32 32 33 34 35 36 37 38 40 40 42 43 45 45 46 47 47 4
7 48 49 49 52 54 57 58 59 60 60 61 61 62 64 64 64 65 65 68 68 68 69 70 71 7
3 74 75 76 76 77 82 82 83 85 85 86 88 89 90 91 91 92 94 96 96 97
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

My Work:

Screenshot of Code

My Code: (problem1_ILOB.h)

ILOB > C problem1_ILOB.h > ...

```
#ifndef PROBLEM1_ILOB_H
#define PROBLEM1_ILOB_H

#include <iostream>

// Partition the array for Quick Sort
template <typename T>
int partition(T arr[], int low, int high) {

    // Select the last element as the pivot
    T pivot = arr[high];

    // Set the position before the first element
    int i = low - 1;

    // Compare each element with the pivot
    for (int j = low; j < high; j++) {

        // Move smaller elements before the pivot
        if (arr[j] < pivot) {
            i++;

            // Swap the elements
            T temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

```

```
    // Place the pivot in its correct position
    T temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    // Return the pivot position
    return i + 1;
}

// Sort the sublist using Bubble Sort
template <typename T>
void bubbleSort(T arr[], int low, int high) {

    // Compare elements in the sublist
    for (int i = low; i < high; i++) {

        for (int j = low; j < high - (i - low); j++) {

            // Swap the elements when the left value is larger
            if (arr[j] > arr[j + 1]) {
                T temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Display all elements in the array
template <typename T>
void displayArr(T arr[], int maxSize) {

    // Display each element in the array
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }

    std::cout << std::endl;
}

#endif
```

My Code: (main_ILOB.cpp)

ILOB > main_ILOB.cpp > main()

```cpp
#include <iostream>
#include "problem1_ILOB.h"

const int maxSize = 7;

int main() {

    // Create an example array
    int arr[maxSize] = {8, 3, 7, 2, 9, 4, 6};

    // Display the original array
    std::cout << "\nArray:" << std::endl;
    std::cout << "------------------------" << std::endl;
    displayArr(arr, maxSize);

    // Partition the array using Quick Sort
    int pivot = partition(arr, 0, maxSize - 1);

    // Display the left sublist
    std::cout << "\nLeft Sublist:" << std::endl;
    std::cout << "------------------------" << std::endl;
    displayArr(arr, pivot);

```

```cpp
    // Display the pivot
    std::cout << "\nPivot: " << arr[pivot] << std::endl;

    // Display the right sublist
    std::cout << "\nRight Sublist:" << std::endl;
    std::cout << "------------------------" << std::endl;
    displayArr(arr + pivot + 1, maxSize - pivot - 1);

    // Sort the left sublist using Bubble Sort
    bubbleSort(arr, 0, pivot - 1);

    // Sort the right sublist using Bubble Sort
    bubbleSort(arr, pivot + 1, maxSize - 1);

    // Display the final sorted array
    std::cout << "\nFinal Sorted Array:" << std::endl;
```

```
40      std::cout << "-------------------------" << std::endl;
41      displayArr(arr, maxSize);
42
43      std::cout << std::endl;
44
45      return 0;
46  }
```

Output

```
Array:
-------------------------
8 3 7 2 9 4 6

Left Sublist:
-------------------------
3 2 4

Pivot: 6

Right Sublist:
-------------------------
9 7 8

Final Sorted Array:
-------------------------
2 3 4 6 7 8 9
```

Observations

The code successfully showed that the ***left*** and ***right sublists*** from the ***Quick Sort partition*** can be sorted using another sorting method. The ***partition(arr, 0, maxSize - 1)*** function divided the values around the pivot, while ***bubbleSort(arr, 0, pivot - 1)*** and ***bubbleSort(arr, pivot + 1, maxSize - 1)*** sorted the left and right sublists separately. The output first showed the original array as ***8 3 7 2 9 4 6***, then the ***left sublist, pivot,*** and ***right sublist*** after the partition process. After applying Bubble Sort to both sublists, the final output showed ***2 3 4 6 7 8 9***, which means the values were successfully arranged from smallest to largest. Overall, the result showed that another sorting algorithm can be used to sort the sublists after the partition process, although checking the separate parts was a little confusing at the start.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- ***For the given array, Quick Sort can give a fast time performance because the average time complexity is O(n log n), which is faster than basic sorting methods like Bubble Sort, Selection Sort, and Insertion Sort that usually take $O(n^2)$ time. Quick Sort works by choosing a pivot and dividing the values into smaller parts, then sorting each part separately. Merge Sort can also have O(n log n) because the array is divided into smaller parts until the values are separated, and then the parts are merged back in sorted order. Quick Sort can reach O(n log n) when the pivot divides the values fairly evenly, while Merge Sort keeps dividing the array into smaller parts and combining them, giving the same overall complexity. For this reason, both can handle larger amounts of data more efficiently, although Quick Sort can still become $O(n^2)$ when the pivot choices are not balanced.***

Conclusion:

The activity helped in learning how different sorting techniques work and how they can be applied using C++. For the lessons learned, Shell Sort compares values that are separated by different gaps, Merge Sort divides the values into smaller parts and combines them in sorted order, while Quick Sort uses a pivot to divide the values into smaller sublists. The activity also showed how the same array of 100 random values can be copied and used for each sorting process so the results can be compared properly. For the procedure, the codes were created and tested step by step, starting with generating and storing random values, then applying Shell Sort, Merge Sort, and Quick Sort. The important parts such as shellSort(), merge(), mergeSort(), partition(), and quickSort() helped show how each sorting method works. For the supplementary activity, the Quick Sort partition process was used with Bubble Sort, where partition(), bubbleSort(), and displayArr() helped divide and sort the left and right sublists, while the second problem helped explain the time performance of the sorting algorithms.

One difficulty faced was understanding the different ways each sorting technique moves and divides the values, especially when checking the gaps in Shell Sort, the merging process in Merge Sort, and the pivot and sublists in Quick Sort. Another difficulty was making sure the copied arrays, code functions, and outputs were all matching, especially when checking the separate sorting results and the supplemental activity. Even with these difficulties, the results were correct and the output showed that the sorting processes worked properly. Overall, the activity was done well because the sorting techniques were successfully implemented and applied to different data problems, but there is still an area for improvement in understanding the differences between each sorting method and tracing the sorting process with less confusion. More practice with arrays, loops, gaps, merging, pivots, and time complexity can help improve the understanding and confidence in using these algorithms.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **formatting, explanation of concepts, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain how the code works by giving a detailed explanation on the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.